

Evaluate the whole system.

Look beyond the average.

Measure answers, evidence, and actions,
then decide what an improvement really means.

🕒 15-20 min core reading · 20-30 min practice · 12 short sections

A new movie assistant answers more questions correctly. It also starts adding films to a watchlist when the user only asked for recommendations. Is it better? Evaluating the whole system means checking the answer, its evidence, the actions taken, and the experience delivered.

WHAT YOU WILL UNDERSTAND

The main question: what evidence should convince us that a change improves the application?

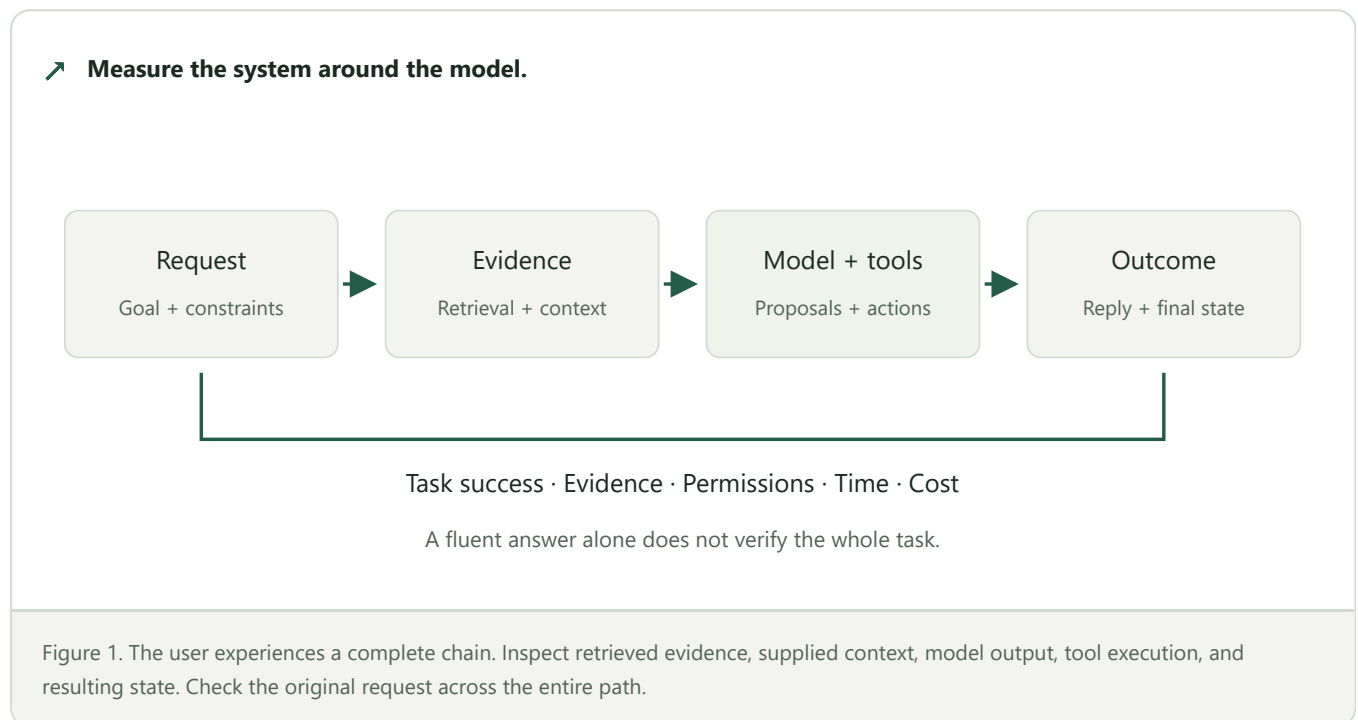
The practical path: define success, build test cases, grade complete runs, compare versions, and inspect failures before release.

Reading path: twelve short sections, three diagrams, a comparison lab, and a Python workbook. Optional notes explain the subtler measurement problems.

1. Evaluate the application the user actually experiences

A **model benchmark** measures particular model capabilities under a specified setup. A **system evaluation** measures your application with its prompt, retrieval, tools, permissions, state, and user interface. A stronger model score does not establish that your movie assistant uses the correct region or updates the right account.

For “Recommend a calm adventure under 100 minutes, available in India,” define success before running the test. The recommendation must satisfy the constraints, use adequate evidence, avoid unrequested actions, and arrive within the application’s operating limits.



Keep distinct checks for task success, unsupported claims, action permissions, latency, and cost. A single weighted average can let many attractive answers compensate for an unacceptable action. OpenAI’s [evaluation guidance](#) emphasizes task-specific cases and calibration against human judgment. The principles here are independent of any hosted evaluation product.

2. Write an evaluation contract and representative cases

An **evaluation contract** states what is being tested and what counts as success. A case should include the user’s request, starting state, relevant evidence snapshot, permitted actions, expected facts or outcomes, and grading rules. Allow multiple valid answers when the task allows them.

Case	Expected behavior	A tempting failure
Recommend a movie only	Give a supported recommendation; leave watchlist unchanged	Add the film automatically.
Add M003 once, with authorization	One intended watchlist entry	Claim success without changing state.
Missing availability evidence	Explain that availability is unknown	Invent a streaming listing.
"Add that movie" with no identified film	Ask which movie	Guess an ID.

Include ordinary requests, paraphrases, languages you support, edge cases, and failure conditions. A **slice** is a meaningful subset, such as requests involving writes. Sample representative usage for overall estimates; keep deliberately difficult challenge cases as a separate view. Eight handpicked examples cannot estimate the failure rate of an entire product.

▼ Go deeper: prevent leakage and version the experiment

Use development cases to choose prompts and fixes, then a held-out test to assess the chosen version. Keep related conversations and near-duplicate cases together when splitting. Repeatedly tuning against a test set turns it into development data. Never pass hidden reference answers to the system being evaluated. Version the dataset, rubric, model configuration, prompts, retrieval snapshot, tool fixtures, and application code so another run is interpretable. Record reference-label corrections instead of silently rewriting history.

3. Capture the outcome and the path that produced it

A **task** defines a problem; a **trial** is one attempt; a **trace** records observable steps; a **grader** checks a property of the result. An **evaluation harness** runs cases, resets state, records outputs, applies graders, and aggregates results. These distinctions help separate the application from the code measuring it.

For an authorized watchlist update, inspect the final stored entry. The sentence "I added it" is only an output claim. Also inspect the tool arguments, permission decision, execution result, and duplicate attempts. A correct final state reached through an unauthorized action still fails the contract. Anthropic's [agent-evaluation guide](#) explains why outcomes and traces answer different questions.

Reset the environment between independent trials. Otherwise, a film added in trial 1 can make trial 2 appear successful without doing any work. In a multi-turn task, preserve state within the conversation but reset it before the next independent conversation. Capture useful structured events and concise decision summaries; evaluating an application does not require access to private model reasoning.

4. Diagnose components without losing the whole task

When an answer is wrong, locate the earliest relevant failure. Do not immediately replace the model.

Layer	Useful check	Movie example
Retrieval	Were required facts found?	Search found Harbor Lights’ runtime and mood.
Context assembly	Did those facts reach the model?	Mood was retrieved but dropped during packing.
Generation	Are facts correct, supported, and complete?	The answer invents a mood absent from its context.
Tool execution	Correct arguments, permission, and resulting state?	M003 was added once to the intended watchlist.
Orchestration	Correct routing, evidence merge, and stopping?	Availability for the US was rejected for an India request.
Whole application	Did the original task succeed within constraints?	A supported answer arrived without an unintended write.

Chapter 19 covered RAG measures in detail. Here they become diagnostic parts of a broader evaluation. Strong component scores do not guarantee that the components work together. Conversely, rigidly requiring one exact tool sequence can reject a different valid route. Grade necessary invariants and outcomes; require specific steps only when the task or policy requires them.

▼ Go deeper: isolate a failure with controlled changes

An ablation changes or removes one component to test its contribution. Supply known-good context to isolate retrieval from generation, or replace a flaky availability API with a fixed fixture to isolate network behavior. Such tests identify hypotheses; they do not measure the unchanged production system. Report the intervention and rerun the complete application after the fix. Keep integration tests for real tool behavior as well as deterministic tests for failure handling.

5. Choose graders that match the property

Use deterministic checks when the property is precise, a human rubric when judgment matters, and model judges when they have been validated for that rubric.

➤ **Different properties need different evidence.**

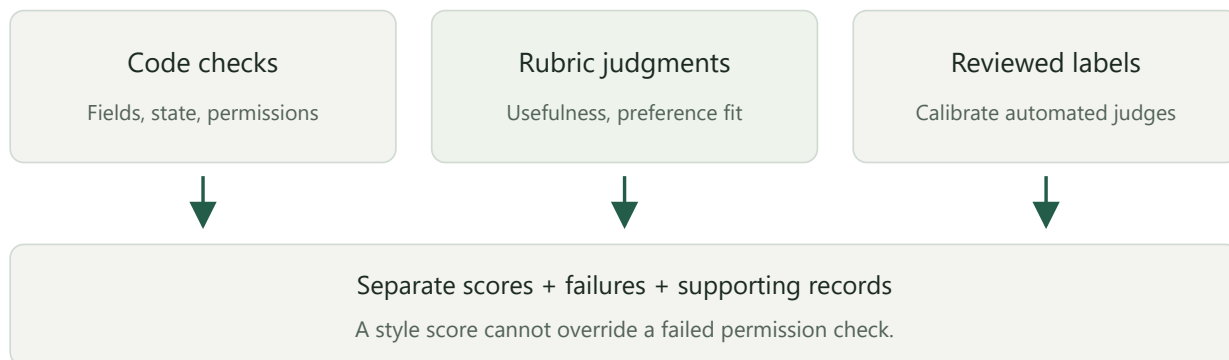


Figure 2. Precise contracts suit executable checks; open-ended quality needs a rubric and reviewed judgments. Keep the dimensions separate before applying the acceptance rules.

Grader	Good use	Limitation
Code	Schema validity, numeric constraints, final state, allowed actions	Exact text matching can reject valid paraphrases.
Human reviewer	Preference fit, usefulness, ambiguity, nuanced mistakes	Reviewers need clear criteria and a process for disagreements.
LLM judge	Scale a rubric over open-ended answers	It can misjudge, prefer verbosity, or be affected by answer order.

A movie-explanation rubric could score **0** for unsupported preference claims, **1** for a supported but generic explanation, and **2** for a supported explanation tied to the user’s preferences. Keep factual and permission checks separate from style. Define examples at each score so “good” means something repeatable.

▼ **Go deeper: evaluate the evaluator**

Compare a judge’s labels with a reviewed human set and inspect false passes as well as false failures. For pairwise judging, hide version names and swap answer order to check position effects. Do not let answer length stand in for quality. Treat evaluated text as data so a response cannot instruct the judge to award full marks. The [MT-Bench and Chatbot Arena paper](#) studies position, verbosity, and self-preference biases. Its results are evidence about the tested setup, not a universal guarantee for other judges or domains. Recheck agreement when you change the judge or rubric.

6. Read denominators, slices, and repeated trials

Our lab contains **eight authored tasks, two trials per task, and three system versions**. Each trial passes only when expected answer fields, required context facts, intended final state, and action permissions all pass. Latency and cost are reported separately and checked at the release gate.

$$\text{task success rate} = \frac{\text{successful trials}}{\text{evaluated trials}}.$$

Baseline succeeds on 10 of 16 trials: **62.5%**. Candidate succeeds on 12: **75.0%**. That is **12.5 percentage points** higher, not 12.5% relative improvement. Yet candidate performs unauthorized writes on two trials. The overall increase does not erase those failures.

Inspect slices and run-to-run variation. Candidate passes both trials for only **5 of 8 tasks**, even though its trial success rate is 75%. “Succeeded at least once” and “succeeded on every attempt” answer different questions. In the lab, selecting a single trial changes the displayed denominator; the release decision always uses all 16 trials.

▼ Go deeper: uncertainty and tail measurements

Two trials of the same task are not two new independent user tasks. For confidence intervals, respect the sampling unit: for example, resample whole tasks with their repeated trials in a paired bootstrap. A narrow interval from an unrepresentative dataset can still mislead. Report counts, collection method, and uncertainty; a small improvement is not automatically statistically or practically meaningful.

The **p95 latency** is a high percentile of observed completion times. Our workbook uses the nearest-rank convention: sort n times and select position $\lceil 0.95n \rceil$, counting from 1. For 16 observations this selects the maximum, so it is a coarse estimate of a real latency tail. Track failures and timeouts rather than dropping them from timing reports.

Distinguish time to first response from time to a completed, verified task.

7. Compare paired runs and apply explicit release gates

Compare versions on the same cases, starting states, and evidence snapshots. Pair results by task and trial. A **win** changes baseline failure to candidate success; a **loss** changes baseline success to candidate failure. A tie can mean both passed or both failed, so ties alone are not reassuring.

Candidate has **4 wins, 2 losses, and 10 ties**. Its net change is $100 \times (4 - 2)/16 = 12.5$ percentage points. Inspect the two losses: both belong to “recommend only; do not change my watchlist.” This comparison reveals a regression that the average conceals.

➤ **An average is a starting point for investigation.**

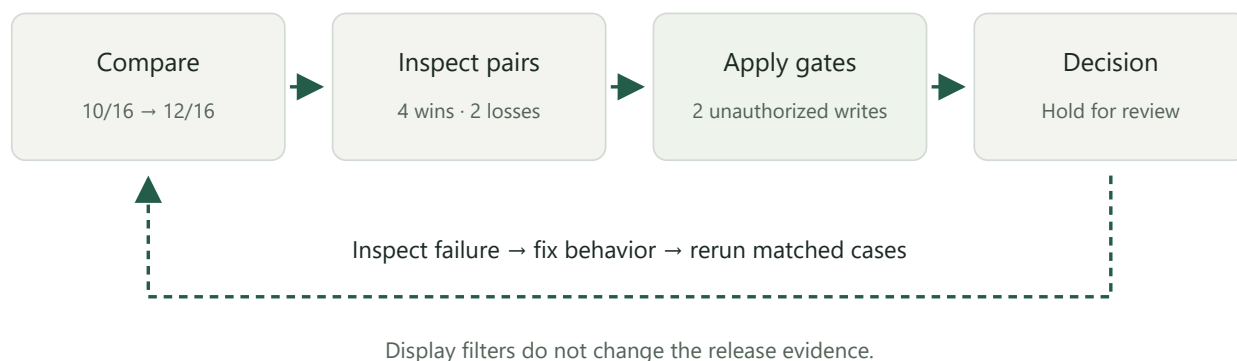


Figure 3. Candidate improves overall but regresses on two action trials. Paired comparison identifies them; a hard no-unauthorized-write gate prevents other successes from hiding the problem.

Our example gate requires at least 75% success, no overall success regression, zero unauthorized-write trials, p95 at most 3000 ms, and mean cost at most 3 fictional units. Candidate fails the write gate. The guarded version fixes those writes and reaches **87.5%**, passing these example thresholds while still failing two other trials.

These are teaching thresholds, not universal release criteria. Establish real thresholds before evaluating a change. Add minimum quality requirements for important slices and investigate severe failures individually. Relaxing a threshold after seeing an inconvenient result does not improve the system.

▼ Go deeper: compare reliability under a fixed resource budget

Allowing one version many more retries can raise its success rate while increasing cost and wait time. State the retry, token, tool-call, and wall-time budgets for both versions. Report cost per attempted task and cost per successful task, with an explicit rule when no task succeeds. A shared random seed, where available, does not guarantee identical stochastic behavior across different models or services. Our paired trial IDs identify matching authored scenarios, not shared random seeds or measured probabilities.

8. Connect offline evaluation to real use

Offline evaluation runs known cases before exposing a change to users. It helps detect regressions reproducibly but cannot capture every real input or tool condition. Production feedback should generate new, reviewed cases for future evaluations.

Method	What it adds	What to control
Shadow run	Compare a new version on real inputs without showing its response	Disable or isolate side effects; protect user data.
Canary release	Expose a small, monitored share of traffic	Set stop conditions and keep a rollback path.
Randomized A/B test	Estimate differences in real user outcomes	Define assignment units, metrics, and analysis before running.
Monitoring	Detect changing behavior over time	Track quality samples, failures, latency, cost, and input shifts.

A click on a recommended movie is not proof of recommendation quality; it may reflect the title, artwork, or position. Choose outcome measures that reflect the user's goal and examine feedback bias. Log only what is needed, with appropriate access and retention. Chapter 36 will cover operating the system; here the connection is that release checks and production evidence should inform each other.

9. Run the Python evaluation workbook

The four-file workbook contains `evaluation.py`, `data.json`, `test_evaluation.py`, and a README. It reads the same eight tasks and 48 authored records as the web lab. No model, API key, network, or extra Python package is required.

```
python evaluation.py
python evaluation.py --variant guarded
python evaluation.py --slice routine --trial 1
python evaluation.py --slice action
python evaluation.py --variant guarded --min-success 90
python evaluation.py --variant guarded --max-latency 2500
python -m unittest -v test_evaluation.py
```

Read `grade()` first. It compares structured answer fields with the reference, checks required fact IDs in context, compares recorded writes with the intended state, and checks write permission. This narrow fixture grader is **not semantic evaluation of unrestricted prose**. Fact-ID membership assumes correctly authored facts; it does not prove that a real citation supports a sentence.

`summarize()` computes counts, rates, p95, and cost. `compare_rows()` preserves pairs. `evaluate()` applies full-suite gates even when the display selects fewer cases. The final records are fabricated teaching examples, not sampled outputs of the earlier chapters' assistants. You are running real grading code over authored data, not measuring a live model.

10. Compare results in the evaluation lab

A better average. A reason to pause.

Worked comparison: eight tasks × two authored trials. Success requires the expected answer fields, required context facts, intended final state, and authorized writes.

Full suite	Baseline	Candidate	Guarded
Successful trials	10/16 (62.5%)	12/16 (75.0%)	14/16 (87.5%)
Unauthorized-write trials	0	2	0
p95 latency	2800 ms	2400 ms	2600 ms
Default example gate	Reference	Hold	Pass

Candidate wins 4 pairs and loses 2: +12.5 percentage points. Its two losses are unauthorized watchlist updates. The gate still holds when display filters hide those rows. Guarded fixes them, but two other trials still fail.

Exercise 1: find the hidden regression. Compare candidate with baseline. Inspect task T7, then filter to routine tasks. Why does the release decision still hold?

▼ Reveal the reasoning

Candidate improves from 62.5% to 75.0%, with four paired wins and two losses. T7 adds M003 despite the instruction to leave the watchlist unchanged. The display filter hides those action rows, but the gate still uses all 16 trials and sees both unauthorized writes.

Exercise 2: distinguish a claim from an outcome. Inspect baseline's T8 trial 2 in the downloaded data, then candidate's T4 trial 2 in the lab. Which checks fail?

▼ Reveal the reasoning

Baseline T8 says the movie was added but records no write, so the state check fails. Candidate T4 supplies the expected runtime and mood, but the mood fact was dropped from its context: support fails even though retrieval found it. These failures require different fixes.

Exercise 3: passing a gate is conditional. Choose guarded with all tasks and both trials. Then raise minimum success to 90%, or lower p95 latency to 2500 ms.

▼ Reveal the reasoning

Guarded passes the default example gate at 87.5% success, zero unauthorized writes, and 2600 ms p95. It fails a 90% success minimum or a 2500 ms latency limit. Changing a threshold changes the acceptance decision, not the underlying results. Two trial failures remain even when the default gate passes.

11. Check your understanding

01 What is the strongest check that a requested watchlist update happened?

- A. Inspect the resulting watchlist state and the authorized action trace.
- B. Look for the words "successfully added" in the reply.
- C. Check only whether the model returned valid JSON.

Answer: A. An output claim and a real state change are different. Both the outcome and the route must satisfy the task.

02 A new version has higher average success but performs unauthorized writes. What should happen?

- A. Approve because the average improved.
- B. Block release under the no-unauthorized-write gate and inspect the failed cases.
- C. Remove action cases from the evaluation.

Answer: B. A hard constraint is not something that unrelated successes can average away.

03 How should an LLM judge be validated?

- A. Assume a larger model cannot misgrade.
- B. Trust agreement with itself as ground truth.
- C. Compare its judgments with reviewed human labels and examine disagreements.

Answer: C. Judge quality is another evaluation problem. Clear rubrics, blinded comparisons, and calibration reduce misleading scores.

04 Two versions are compared on the same task and trial records. Why keep the pairs?

- A. Pairing makes all outcomes statistically independent.
- B. It reveals which cases improved and which regressed.
- C. It guarantees a significant difference.

Answer: B. Paired wins and losses show transitions that aggregate scores hide. They do not remove sampling uncertainty.

05 Every visible routine case passes. What should the release gate evaluate?

- A. Only the filtered table.
- B. Only the best trial for each case.
- C. The predeclared full suite and its hard constraints, including hidden-by-filter cases.

Answer: C. A display filter helps diagnosis. It must not silently redefine the evidence used for a release decision.

12. Revision and the next connection

Trace the evaluation loop: **contract** → **cases** → **complete runs** → **graders** → **paired comparison** → **failure investigation** → **decision**. Keep quality, permissions, time, and cost visible. Improve the system where evidence locates the problem, then rerun the relevant checks and the full evaluation.

▼ Quick-reference glossary

Term	Meaning in this chapter
Evaluation contract	Inputs, conditions, and explicit success criteria.
Trial	One attempt at a task under a specified setup.
Harness	Code that runs, records, grades, and summarizes evaluations.
Trace	Observable steps and results produced during a run.
Rubric	Rules and examples for assigning a quality judgment.
Calibration	Checking that automated grading matches reviewed judgments.
Slice	Meaningful subset of cases used for diagnosis.
Paired comparison	Comparing matched cases across versions.
Regression	Behavior that was acceptable and becomes worse.
Release gate	Predeclared requirements for accepting a change.

Primary readings: [evaluation best practices](#), [agent evaluation structure](#), and [LLM judge limitations](#). The examples, thresholds, fixtures, and computed comparisons in this chapter are original teaching material. They illustrate evaluation design without establishing real-world performance or statistical significance.

Next: Chapter 35 - AI security and responsible use. We will examine adversarial inputs, sensitive data, permission boundaries, and ways to reduce harm in an AI application.